

```

1      .TITLE  MANDELBROT      ASCII Mandelbrot (user W x H) written to MANDELBROT.TXT
2      .IDENT  /V1.2/
3
4  ;++
5  ; MANDELBROT.MAR
6  ;
7  ; Prompts the user for a width (columns) and height (rows) on
8  ; SYS$INPUT, then renders the Mandelbrot set as ASCII art at that
9  ; resolution into the file MANDELBROT.TXT in the current default
10 ; directory -- one record (one line) per row.
11 ;
12 ; Width is capped at MAX_W (512) -- the static row buffer size.
13 ; Height is capped at MAX_H (4096) -- a sanity bound.
14 ;
15 ; Prompts and any error messages still go to SYS$OUTPUT so the user
16 ; can interact; only the rendered image is written to the file.
17 ;
18 ; On invalid input (non-digit, zero, or out-of-range) a short error
19 ; message is printed and the image exits with SS$_BADPARAM.
20 ;
21 ; Algorithm (unchanged):
22 ;
23 ; For each pixel (col, row):
24 ;     c = Cr + Ci*i          ; linear map from pixel coords to
25 ;                             ; REAL_MIN..REAL_MAX / IMAG_MIN..IMAG_MAX
26 ;     z = 0; iter = 0
27 ;     while iter < MAX_ITER:
28 ;         if |z|^2 > 4: break      ; escaped
29 ;         z = z*z + c
30 ;         iter += 1
31 ;     plot GRADIENT[iter/3] or INSET_CHAR if iter == MAX_ITER
32 ;
33 ; Inline squaring:
34 ;     Zr_new = Zr^2 - Zi^2 + Cr
35 ;     Zi_new = 2*Zr*Zi + Ci
36 ;     Escape test compares squared magnitude (Zr^2 + Zi^2 > 4) -- no sqrt.
37 ;
38 ; Output file characteristics:
39 ; FNM = SYS$DISK:[ ]MANDELBROT.TXT (current default directory)
40 ; FAC = PUT
41 ; ORG = SEQ
42 ; RFM = VAR (variable-length records)
43 ; RAT = CR (carriage-return carriage control -- one record = one line)
44 ; MRS = MAX_W (max record size; actual RSZ patched at runtime)
45 ;
46 ; Error handling:
47 ; - LIB$GET_INPUT / OTS$CVT_TU_L failure -> RET with R0.
48 ; - WIDTH / HEIGHT out of bounds -> message + SS$_BADPARAM.
49 ; - $CREATE failure -> RET with R0 (no cleanup needed).
50 ; - $CONNECT / $PUT failure -> best-effort $CLOSE, return R0.
51 ;
52 ; References consulted:
53 ; reference/VAX-VMS-731/vax-macro-ref-2001.md
54 ; Chapter 9 -- ADDF2/3, SUBF3, MULF3, DIVF3, CMPF, CLRF, CVTLF,
55 ; DIVL3, MOVW, conditional branches.
56 ; Chapter 5 -- indexed addressing label[Rn] scaled by operand size.
57 ; reference/VAX-VMS-731/rtl-lib.md
58 ; LIB$GET_INPUT, LIB$PUT_OUTPUT.
59 ; reference/VAX-VMS-731/rtl-ots.md
60 ; OTS$CVT_TU_L.
61 ; reference/VAX-VMS-731/rms-reference.md
62 ; $FAB/$RAB keyword args; $CREATE/$CONNECT/$PUT/$CLOSE.

```

File - /Users/msw/code/retro/macro-32/src/macro32/mandelbrot/mandelbrot.mar

```
63 ;      RAB$W_RSZ field offset (set at runtime so one FAB/RAB serves
64 ;      any WIDTH up to MAX_W).
65 ;      reference/macro-made-easy/part-09-using-rms-from-macro.md
66 ;      RMS fundamentals, $FABDEF/$RABDEF.
67 ;
68 ; Build (on OpenVMS):
69 ;   $ MACRO MANDELBROT
70 ;   $ LINK MANDELBROT
71 ;   $ RUN MANDELBROT
72 ;   $ TYPE MANDELBROT.TXT
73 ;--
74
75      MAX_W          = 512                ; row buffer limit
76      MAX_H          = 4096              ; sanity limit on rows
77      MAX_ITER       = 30                ; must equal 3 * GRADIENT_LEN
78      GRADIENT_LEN   = 10
79      IN_MAX         = 80
80
81 ;
82 ; Symbol macro includes.
83 ;
84      $DSCDEF        ; DSC$K_DTYPE_T, DSC$K_CLASS_S
85      $SSDEF         ; SS$_BADPARAM, SS$_NORMAL
86      $FABDEF        ; FAB$* symbols used by $FAB macro
87      $RABDEF        ; RAB$W_RSZ and friends
88
89 ;
90 ; Data.
91 ;
92      .PSECT MANDEL_DATA, NOEXE, WRT, LONG
93
94 ; User-supplied grid dimensions (parsed from input).
95 WIDTH:      .LONG  0
96 HEIGHT:     .LONG  0
97
98 ; View window edges, F_floating.
99 REAL_MIN:   .F_FLOATING -2.5
100 REAL_MAX:   .F_FLOATING 1.0
101 IMAG_MIN:   .F_FLOATING -1.2
102 IMAG_MAX:   .F_FLOATING 1.2
103 FOURF:      .F_FLOATING 4.0                ; escape-radius squared
104
105 ; Per-pixel step sizes, derived at runtime from WIDTH / HEIGHT.
106 DX:         .F_FLOATING 0.0
107 DY:         .F_FLOATING 0.0
108
109 ; Input line buffer + descriptor + resultant-length word.
110 IN_BUF:      .BLKB  IN_MAX
111 IN_LEN:      .WORD  0
112
113      .ALIGN LONG
114 IN_DESC:     .WORD  IN_MAX
115              .BYTE  DSC$K_DTYPE_T
116              .BYTE  DSC$K_CLASS_S
117              .ADDRESS IN_BUF
118
119 ; Prompt and error descriptors (all go to the terminal via LIB$PUT_OUTPUT
120 ; or LIB$GET_INPUT's built-in prompt support -- never to the file).
121 INTRO_1:     .ASCII  /VAX MACRO-32 ASCII Mandelbrot renderer./
122 INTRO_2:     .ASCII  /Output will be written to MANDELBROT.TXT in the current directory./
123 DONE_MSG:    .ASCII  /Done. Output written to MANDELBROT.TXT./
124
```

```

125 PROMPT_W:      .ASCID  /Width (1-512): /
126 PROMPT_H:      .ASCID  /Height (1-4096): /
127 ERR_W:          .ASCID  /Width out of range (must be 1..512)./
128 ERR_H:          .ASCID  /Height out of range (must be 1..4096)./
129
130 ; Row buffer (sized to MAX_W). The RAB's RBF points here; RSZ is patched
131 ; to the actual WIDTH once the user has been prompted.
132 ROW_BUF:        .BLKB   MAX_W
133
134 ; Shade ramp. 10 chars: lightest (fast escape) -> darkest (slow escape).
135 ; Points in the set use INSET_CHAR.
136 GRADIENT:        .ASCII  / .,:-+*#%/
137 INSET_CHAR       = ^A/@/
138
139 ; RMS control blocks -- must be longword aligned.
140      .ALIGN      LONG
141 OUT_FAB:          $FAB    FNM=<SYS$DISK:[]MANDELBROT.TXT>,-
142                  FAC=PUT,-
143                  ORG=SEQ,-
144                  RFM=VAR,-
145                  RAT=CR,-
146                  MRS=MAX_W
147
148      .ALIGN      LONG
149 OUT_RAB:          $RAB    FAB=OUT_FAB,-
150                  RAC=SEQ,-
151                  RBF=ROW_BUF,-
152                  RSZ=MAX_W
153
154 ;
155 ; Code.
156 ;
157      .PSECT      MANDEL_CODE, EXE, NOWRT, LONG
158
159 ;
160 ; MANDELBROT -- program entry point.
161 ;
162 ;   Integer registers:
163 ;       R2 = current row
164 ;       R3 = current column
165 ;       R4 = iteration counter
166 ;   F_floating registers:
167 ;       R5 = Zr    R6 = Zi    (current z)
168 ;       R7 = Cr    R8 = Ci    (current c)
169 ;       R9 = Zr^2  R10 = Zi^2
170 ;       R11 = scratch / Zi_new buffer
171 ;
172      .ENTRY      MANDELBROT, ^M<R2,R3,R4,R5,R6,R7,R8,R9,R10,R11>
173
174 ; -----
175 ; Banner. We don't check R0 -- if SYS$OUTPUT is broken everything else
176 ; will fail anyway.
177 ; -----
178      PUSHAL      INTRO_1
179      CALLS       #1, G^LIB$PUT_OUTPUT
180      PUSHAL      INTRO_2
181      CALLS       #1, G^LIB$PUT_OUTPUT
182
183 ; -----
184 ; Phase 1: prompt for width, parse, validate.
185 ; -----
186      PUSHAW      IN_LEN

```

```

187     PUSHAL    PROMPT_W
188     PUSHAL    IN_DESC
189     CALLS     #3, G^LIB$GET_INPUT
190     BLBS      R0, 10$
191     RET
192 10$:
193     MOVW      IN_LEN, IN_DESC                ; shrink to actual bytes
194     PUSHAL    WIDTH
195     PUSHAL    IN_DESC
196     CALLS     #2, G^OTS$CVT_TU_L
197     BLBS      R0, 20$
198     RET
199 20$:
200     TSTL      WIDTH
201     BGTR      21$
202     BRW       WIDTH_ERR
203 21$:
204     CMPL      WIDTH, #MAX_W
205     BLEQ      22$
206 22$:
207
208 ; -----
209 ; Phase 2: prompt for height, parse, validate.
210 ; -----
211     MOVW      #IN_MAX, IN_DESC                ; restore descriptor size
212     PUSHAW    IN_LEN
213     PUSHAL    PROMPT_H
214     PUSHAL    IN_DESC
215     CALLS     #3, G^LIB$GET_INPUT
216     BLBS      R0, 30$
217     RET
218 30$:
219     MOVW      IN_LEN, IN_DESC
220     PUSHAL    HEIGHT
221     PUSHAL    IN_DESC
222     CALLS     #2, G^OTS$CVT_TU_L
223     BLBS      R0, 40$
224     RET
225 40$:
226     TSTL      HEIGHT
227     BGTR      41$
228     BRW       HEIGHT_ERR
229 41$:
230     CMPL      HEIGHT, #MAX_H
231     BLEQ      42$
232 42$:
233
234 ; -----
235 ; Phase 3: create and open MANDELBROT.TXT, patch RSZ to actual WIDTH.
236 ; -----
237     $CREATE   FAB=OUT_FAB
238     BLBS      R0, 50$
239     RET                ; no file to clean up on failure
240 50$:
241     $CONNECT  RAB=OUT_RAB
242     BLBS      R0, 60$
243 ; $CONNECT failed after a successful $CREATE -- close + return R0.
244     PUSHL     R0
245     $CLOSE    FAB=OUT_FAB
246     POPL      R0
247     RET
248 60$:

```

```

249 ; Patch RAB.RSZ with actual WIDTH (low word of WIDTH is the byte count).
250     MOVW    WIDTH, OUT_RAB+RAB$W_RSZ
251
252 ; -----
253 ; Phase 4: compute DX, DY.
254 ; -----
255     SUBF3    REAL_MIN, REAL_MAX, R5           ; R5 = 3.5
256     CVTLF    WIDTH, R6
257     DIVF3    R6, R5, DX                       ; DX = 3.5 / WIDTH
258
259     SUBF3    IMAG_MIN, IMAG_MAX, R5          ; R5 = 2.4
260     CVTLF    HEIGHT, R6
261     DIVF3    R6, R5, DY                       ; DY = 2.4 / HEIGHT
262
263     CLRL     R2                               ; row = 0
264
265 ; -----
266 ; Outer row loop.
267 ; -----
268 ROW_LOOP:
269 ; Ci = IMAG_MIN + row * DY
270     CVTLF    R2, R8
271     MULF2    DY, R8
272     ADDF2    IMAG_MIN, R8
273
274     CLRL     R3                               ; col = 0
275
276 ; -----
277 ; Inner column loop.
278 ; -----
279 COL_LOOP:
280 ; Cr = REAL_MIN + col * DX
281     CVTLF    R3, R7
282     MULF2    DX, R7
283     ADDF2    REAL_MIN, R7
284
285 ; Reset orbit.
286     CLRF     R5                               ; Zr = 0
287     CLRF     R6                               ; Zi = 0
288     CLRL     R4                               ; iter = 0
289
290 ; -----
291 ; Escape-time iteration.
292 ; -----
293 ITER_LOOP:
294     MULF3    R5, R5, R9                       ; R9 = Zr^2
295     MULF3    R6, R6, R10                      ; R10 = Zi^2
296     ADDF3    R9, R10, R11                     ; R11 = |z|^2
297     CMPF     R11, FOURF
298     BGTR     ESCAPED
299
300 ; z_new = z*z + c -- Zi first (needs old Zr,Zi), then Zr.
301     MULF3    R5, R6, R11                     ; R11 = Zr*Zi
302     ADDF2    R11, R11                         ; R11 = 2*Zr*Zi
303     ADDF2    R8, R11                         ; R11 = 2*Zr*Zi + Ci = Zi_new
304
305     SUBF3    R10, R9, R5                     ; R5 = Zr^2 - Zi^2
306     ADDF2    R7, R5                           ; R5 = Zr^2 - Zi^2 + Cr = Zr_new
307
308     MOVF     R11, R6                         ; R6 = Zi_new
309
310     INCL     R4

```

```

311      CMPL    R4, #MAX_ITER
312      BLSS    ITER_LOOP
313
314 ; Fell through -> point is in the set.
315      MOVVB   #INSET_CHAR, ROW_BUF[R3]
316      BRB     NEXT_COL
317
318 ; Escape -> pick from shade ramp.
319 ESCAPED:
320      DIVL3    #3, R4, R1                ; R1 = iter / 3 in [0..9]
321      MOVVB    GRADIENT[R1], ROW_BUF[R3]
322
323 NEXT_COL:
324      INCL     R3
325      CMPL     R3, WIDTH
326      BLSS     COL_LOOP
327
328 ; End of row: write the buffer as one RMS record.
329      $PUT     RAB=OUT_RAB
330      BLBC     R0, PUT_ERR                ; $PUT failed -> close + return
331
332      INCL     R2
333      CMPL     R2, HEIGHT
334 ; Invert + BRW: row body exceeds byte-branch range.
335      BGEQ     DONE
336      BRW     ROW_LOOP
337
338 ; -----
339 ; Normal exit: close the file, then tell the user where the output went.
340 ; If $CLOSE fails we skip the "Done" message and return its error status.
341 ; -----
342 DONE:   $CLOSE  FAB=OUT_FAB
343         BLBS    R0, 70$
344         RET
345 70$:    PUSHAL  DONE_MSG
346         CALLS   #1, G^LIB$PUT_OUTPUT
347         RET
348
349 ; -----
350 ; $PUT failure inside the render loop -- preserve R0, best-effort close.
351 ; -----
352 PUT_ERR:
353         PUSHL   R0
354         $CLOSE  FAB=OUT_FAB
355         POPL    R0
356         RET
357
358 ; -----
359 ; Input-validation error handlers (no file is open yet).
360 ; -----
361 WIDTH_ERR:
362         PUSHAL  ERR_W
363         CALLS   #1, G^LIB$PUT_OUTPUT
364         MOVL    #SS$_BADPARAM, R0
365         RET
366
367 HEIGHT_ERR:
368         PUSHAL  ERR_H
369         CALLS   #1, G^LIB$PUT_OUTPUT
370         MOVL    #SS$_BADPARAM, R0
371         RET
372

```

File - /Users/msw/code/retro/macro-32/src/macro32/mandelbrot/mandelbrot.mar

```
373      .END      MANDELBROT
374
```